

Documento de Visão:

Sistema de Gestão Financeira Pessoal

2. Documento de Visão

1. Contexto e problema Muitos jovens adultos e estudantes enfrentam dificuldades no planejamento financeiro a médio e longo prazo. A ausência de uma ferramenta que integre o rastreamento diário de despesas com mecânicas de projeção de orçamentos dificulta a poupança para projetos futuros, como viagens internacionais, resultando em desorganização e perda de poder de compra devido a variações cambiais.

2. Justificativa A implementação de uma plataforma digital centralizada de gestão financeira permite que o usuário aplique regras claras de distribuição de rendimentos (como a alocação de orçamentos em cascata). Isso proporciona previsibilidade, facilita o acompanhamento de contas em diferentes moedas e assegura que as metas de poupança sejam atingidas de forma sustentável e controlada.

3. Objetivos

- Proporcionar um controle rigoroso e categorizado de receitas e despesas.
- Automatizar o planejamento através de regras de alocação de orçamento em cascata.
- Facilitar o acompanhamento de metas financeiras internacionais (ex: viagens aos Estados Unidos), consolidando valores em contas dolarizadas em tempo real.

4. Público-alvo

- Jovens adultos, estudantes universitários e profissionais em início de carreira que necessitam organizar suas finanças pessoais.
- Pessoas com objetivos de poupança a médio e longo prazo, nomeadamente para intercâmbios ou viagens internacionais.

5. Stakeholders

- Equipe de Desenvolvimento (alunos responsáveis pelo projeto).
- Professor Avaliador (patrocinador do projeto).
- Usuários Finais (pessoas em busca de organização financeira).

6. Escopo A aplicação será um sistema web focado na gestão das finanças pessoais, contemplando:

- Módulo de registro e categorização de despesas e receitas.
- Dashboard interativo para visualização da saúde financeira e consolidação de gastos.
- Sistema de definição de metas de poupança, com consumo de uma API externa de cotação de moedas (para cálculo do câmbio atualizado de moedas estrangeiras, como o Dólar).
- API REST própria para disponibilizar o resumo financeiro mensal do usuário.

7. Itens fora do escopo

- Integração com o sistema Open Banking (ligação direta aos bancos para importar

extratos automaticamente).

- Realização de transferências bancárias ou pagamento de faturas diretamente pela plataforma.
- Aconselhamento e corretagem de investimentos na bolsa de valores.

8. Funcionalidades

- Cadastro, alteração, consulta e exclusão (CRUD) de transações financeiras e categorias.
- Busca avançada de transações com filtros por data, categoria e valor.
- Geração de relatórios mensais e dashboards interativos demonstrando a alocação de fundos.
- Integração com API externa de câmbio (ex: *AwesomeAPI*) para converter o valor atual das metas de viagem.
- Criação de rotas da própria API REST para exportar os dados consolidados do orçamento.

9. Restrições

- O backend da aplicação deverá ser desenvolvido em Python, utilizando o framework Django e suas boas práticas.
- A persistência de dados deverá ser feita em um banco de dados relacional compatível (SQLite para o desenvolvimento inicial).
- O histórico de desenvolvimento tem de ser mantido no GitHub, utilizando o template obrigatório da disciplina.
- Nenhuma informação sensível (senhas, chaves de API, arquivos .env) pode ser versionada no repositório público.

10. Premissas

- Os usuários possuem conhecimentos básicos de educação financeira para operar a plataforma.
- A API de cotação de moedas escolhida se manterá gratuita e funcional ao longo do período de avaliação.
- O usuário acessará o sistema através de um navegador web atualizado, em um computador ou dispositivo móvel.

11. Riscos iniciais

- Dificuldade inicial na modelagem dos bancos de dados no Django ORM para a mecânica de alocação em cascata.
- Latência ou indisponibilidade da API externa de câmbio durante a apresentação, podendo exigir a implementação de dados falsos (mockados) por precaução.
- Desalinhamento do tempo de desenvolvimento da interface responsiva em comparação com a construção do backend.

12. Critérios de sucesso

- Aprovação integral do documento de visão e da arquitetura do projeto na Fase 1.
- Consistência verificável entre esta documentação de projeto e o sistema construído na Fase 2.
- Sistema funcional entregue no prazo, implementando no mínimo as operações de CRUD, relatórios, integração externa e API própria, rodando de forma estável.

6. Contrato inicial da API

Este documento define o contrato do endpoint principal da aplicação, responsável por cruzar as metas financeiras do usuário com as taxas de câmbio atualizadas.

Recursos expostos: Consulta de metas de poupança com conversão cambial

Endpoint: /api/v1/metast/conversao-dolar/

Métodos HTTP: GET

Autenticação: Requerida via Token JWT (Header: Authorization: Bearer).

Parâmetros: Incluir_histórico=true (Query String, opcional).

Códigos de Status:

200 OK: Sucesso.

401 Unauthorized: Usuário não autenticado ou token inválido.

502 Bad Gateway: Erro de comunicação com a API externa (AwesomeAPI).

Exemplo de Requisição:

```
GET /api/v1/metast/conversao-dolar/ HTTP/1.1
Host: api.flowcash.com
Authorization: Bearer eyJhbGciOiJIUzI1Ni...
```

Exemplo de Resposta JSON:

```
{
  "cotacao_atual": {
    "moeda": "USD",
    "valor_brl": 5.45,
    "fonte": "AwesomeAPI",
    "data_consulta": "2026-10-08T16:24:00Z"
  },
  "metas_dolarizadas": [
    {
      "id_meta": 1,
      "titulo": "Viagem aos Estados Unidos",
      "valor_alvo_usd": 3500.00,
      "progresso_atual_brl": 9537.50,
      "progresso_convertido_usd": 1750.00,
      "percentagem_concluida": 50.0
    }
  ]
}
```

7. Plano de Integração Externa

Para cumprir o requisito de integração com serviços de terceiros e agregar valor à funcionalidade de gestão de metas internacionais, o sistema fará chamadas a uma API de conversão de moedas.

API Escolhida: AwesomeAPI (Cotações de Moedas).

Finalidade: Obter a taxa de câmbio atualizada em tempo real, permitindo que o sistema converta as poupanças do utilizador e calcule com precisão se a meta financeira internacional já foi atingida.

Endpoints Consumidos: GET

https://economia.awesomeapi.com.br/last/moeda-moeda_base

Dados Utilizados: O sistema extrairá o valor de compra atual da moeda (campo bid) e a data/hora da última atualização (campo create_date).

Limites e Autenticação: A AwesomeAPI é um serviço público e gratuito. Não exige o registo de chaves de autenticação (API Key) ou tokens no cabeçalho das requisições. O limite de requisições é suficientemente tolerante para o volume de acessos previsto no escopo académico, não havendo documentação de quotas estritas.

Tratamento de Indisponibilidade: Em caso de falha de comunicação (Timeout) ou erro por parte do servidor externo (Status 5xx), o backend em Django implementará um mecanismo de fallback. O sistema recorrerá à última cotação que foi guardada em banco de dados na consulta anterior (cache local) e exibirá um alerta visual ao utilizador informando que os valores apresentados baseiam-se numa cotação em cache devido à indisponibilidade temporária do serviço de câmbio.

Referência à documentação oficial: <https://docs.awesomeapi.com.br/api-de-moedas>

8) Identidade visual e protótipos

8.1 Diretrizes de Identidade Visual (prontas para colar no doc)

Objetivo

Criar uma identidade visual consistente e legível para um **Sistema de Gestão Financeira**, transmitindo **confiança, clareza e controle**.

Paleta de cores (exemplo profissional)

- **Azul (primário):** #0B5FFF
- **Verde (positivo/receitas):** #14B86A
- **Vermelho (alertas/despesas/erros):** #E53935
- **Cinza (neutros):** #6B7280
- **Fundo (modo claro):** #F8FAFC
- **Texto principal:** #111827

Se quiser, posso adaptar para a marca da sua empresa (basta mandar o logo/cores).

Tipografia (recomendada)

- **Títulos:** Inter (Semibold/Bold)

- **Corpo:** Inter (Regular/Medium)

Estilo de UI (padrão)

- Componentes com **cantos levemente arredondados** (ex: radius 8px)
- Botões com **hierarquia clara**:
 - Primário (azul)
 - Secundário (cinza)
 - Perigo/Desativado (vermelho/cinza claro)
- Espaçamento consistente (grid 8px)

Ícones e linguagem visual

- Estilo de ícones: **line icons** (simples, consistentes)
 - Linguagem: verbos curtos
 - “Cadastrar”, “Atualizar”, “Filtrar”, “Exportar”, “Lançar”
-

8.2 Protótipos de Telas

Abaixo estão **telas recomendadas** para o protótipo. Você pode implementar em Figma ou exportar imagens para o repositório.

Tela 1 — Login

Elementos:

- Logo (topo)
- Campos: Email / Senha
- Botões:
 - “Entrar” (primário)
 - “Esqueci minha senha” (link)
- Mensagem de erro (ex.: “Usuário ou senha inválidos”)

Tela 2 — Dashboard Financeiro

Seções:

- Cards:
 - **Saldo atual**
 - **Receitas do mês**
 - **Despesas do mês**
 - **Lucro/Saldo do período**
- Gráficos:
 - Linha: “Movimentação por dia/semana”

- Barra: “Receitas vs Despesas por categoria”
- Tabela rápida:
 - “Últimos lançamentos”
 - Colunas: Data | Descrição | Categoria | Valor | Tipo

Tela 3 — Lançamentos (Lista + Filtros)

Funcionalidades:

- Barra de busca
- Filtros:
 - Período (início/fim)
 - Tipo (receita/despesa)
 - Categoria
 - Status (se aplicável)
- Ações por linha:
 - Editar | Excluir
- Botão “+ Novo Lançamento”

Tela 4 — Criar/Editar Lançamento

Campos:

- Tipo (receita/despesa) — toggle ou select
- Data
- Descrição
- Categoria
- Valor (R\$)
- Forma de pagamento (cartão/boleto/PIX/etc, se existir no sistema)
- Observações (opcional)
- Botões:
 - Salvar (primário)
 - Cancelar (secundário)

Tela 5 — Categorias

Funcionalidades:

- Lista de categorias
- Ações: editar/excluir
- Botão “+ Nova Categoria”
- Campos:
 - Nome
 - Tipo (receita/despesa) *se for necessário*

Tela 6 — Relatórios

Módulos:

- “Resumo por período”
- “Por categoria”
- “Fluxo de caixa” (se aplicável) **Ações:**
- Selecionar período
- Gerar relatório
- Exportar (PDF/CSV) — mesmo que seja mock no protótipo

Tela 7 — Perfil / Configurações

- Dados do usuário
 - Trocar senha
 - Preferências (ex.: moeda padrão)
-

8.3 Arquivos que você deve criar dentro de

Sugestão (nomes prontos):

- docs/prototipos/01_identidade_visual.md
- docs/prototipos/02_palette_e_fonts.png (ou .svg)
- docs/prototipos/03_prototipo_login.png
- docs/prototipos/04_prototipo_dashboard.png
- docs/prototipos/05_prototipo_lancamentos.png
- docs/prototipos/06_prototipo_form_lancamento.png
- docs/prototipos/07_prototipo_categorias.png
- docs/prototipos/08_prototipo_relatorios.png
- docs/prototipos/09_prototipo_configuracoes.png

*Se você me disser se vai usar **Figma** ou **diagrams.net**, eu adapto o formato e o “layout” para ficar compatível.*

9) Planejamento

9.1 Escopo do Planejamento

Objetivo: organizar atividades para o desenvolvimento do **Sistema de Gestão Financeira** com entregas claras.

9.2 Roadmap sugerido (MVP → Evolução)

Fase 1 – MVP (primeira entrega)

- Autenticação (login + cadastro se necessário)
- CRUD de **Lançamentos**
- Cadastro de **Categorias**
- Dashboard com métricas básicas
- Relatórios simples (por período e por categoria)

Fase 2 – Evolução

- Exportação (CSV/PDF)
 - Melhoria de gráficos e filtros avançados
 - Fluxo de caixa / previsões (se aplicável)
 - Ajustes de UX e permissões avançadas
-

9.3 Cronograma (modelo em semanas)

Ajuste para sua duração real. Exemplo de 6 semanas:

- **Semana 1:** levantamento + identidade visual + wireframes
 - **Semana 2:** modelagem + setup do projeto + telas base
 - **Semana 3:** autenticação + backend do CRUD de lançamentos
 - **Semana 4:** categorias + dashboard + filtros
 - **Semana 5:** relatórios + exportação (mock se necessário)
 - **Semana 6:** refinamento geral + testes + documentação
-

9.4 Backlog (tabela pronta)

Prioridade alta (MVP)

- Tela Login
- Tela Dashboard (cards + tabela últimos lançamentos)
- Tela Lançamentos (listar + filtros + ações)
- Form Lançamento (criar/editar)
- Categorias (CRUD)
- Relatórios (gerar por período/categoria)

Prioridade média

- Exportação (CSV/PDF)
- Validações mais completas
- UI polish (estados vazios, loaders)

Prioridade baixa

- Alertas/insights (ex.: “despesas acima do normal”)
- Previsões

9.5 Riscos e Mitigações (pronto para doc)

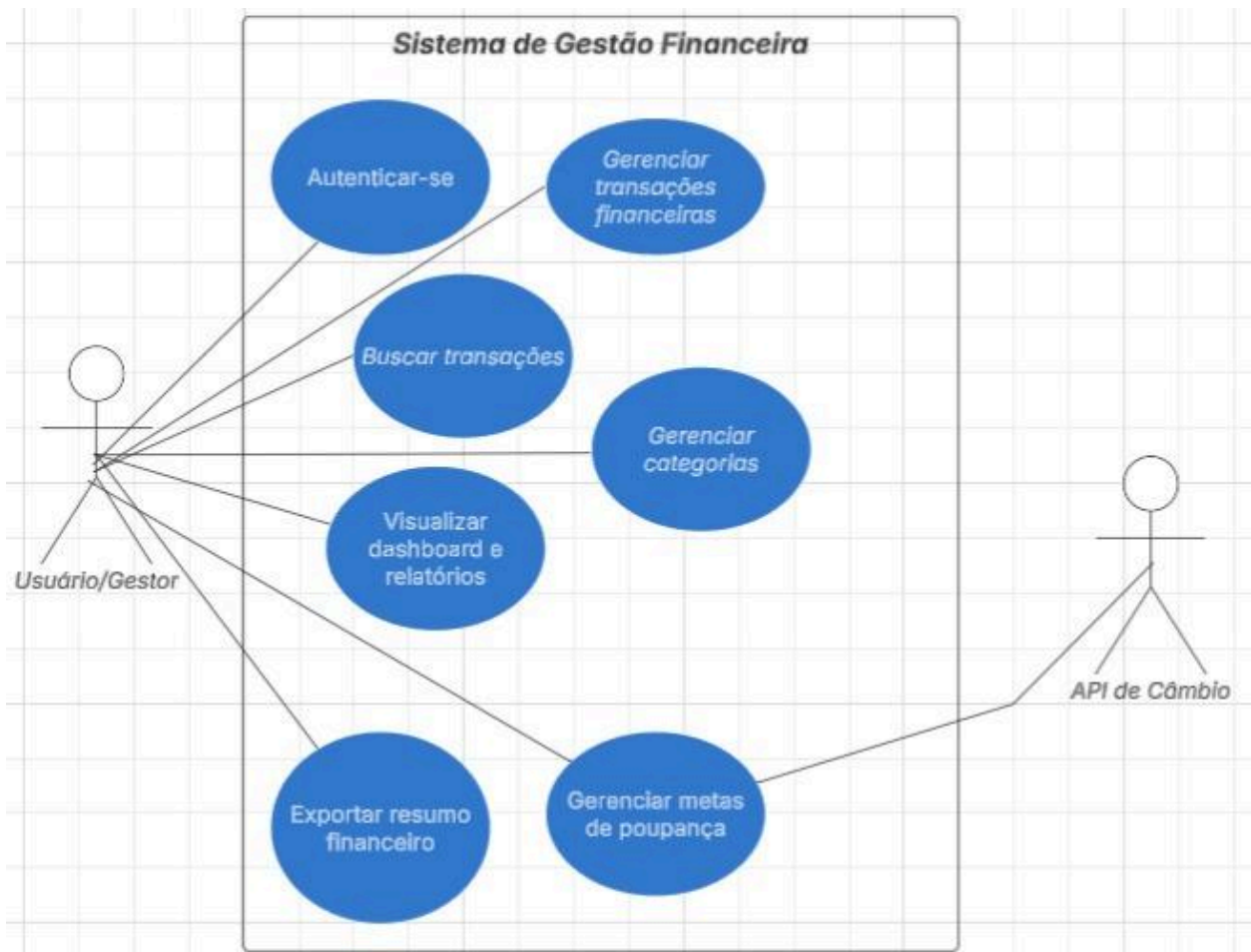
- **Risco:** escopo crescendo (muchas features)
Mitigação: travar MVP e fazer releases por fase.
- **Risco:** falta de consistência visual entre telas
Mitigação: usar paleta/tipografia e componentes padrão.
- **Risco:** dados inconsistentes em lançamentos (valores/categorias)
Mitigação: validações + regras no backend.

9.6 Arquivos que você deve criar dentro de docs/planejamento/

Sugestão (nomes prontos):

- docs/planejamento/01_roadmap.md
- docs/planejamento/02_cronograma_semana_a_semana.md
- docs/planejamento/03_backlog_mvp.md
- docs/planejamento/04_riscos_mitigacoes.md

10 Diagrama de Caso de Uso



https://lucid.app/lucidchart/8a1a6a93-5e1c-410b-a3af-dc5e35b3e2df/edit?viewport_loc=-1555%2C-387%2C1587%2C763%2C0_0&invitationId=inv_9c82b5c0-d6b9-4804-b0e4-340e1a520155